

Python for Data Science

PROGRAM	ASSESSMENT CODE	TOTAL TIME	STACK
Data Science	M2-A1	3.5 Hours	Python

GENERAL INSTRUCTIONS

Attempt all sections. This is a cumulative assessment covering Module 1 (Introduction to Data Science & Git/GitHub) and Module 2 (Python for Data Science) only — no other module's concepts are in scope. Git tasks must be performed in a Bash terminal or Git Bash on Windows — submit a .txt file with the command sequence and a terminal screenshot. Python tasks must be submitted as a .py source file with a screenshot of the program output. For Section A, write structured prose answers of at least 6–8 lines per question. Label every answer clearly with its section and task number. All work must be your own.

A Section A — Concept Application

Answer all questions. Questions 1–2 draw from Module 1 (Introduction to Data Science & Git/GitHub); Questions 3–6 draw from Module 2 (Python for Data Science). Each question presents a real-world scenario — your answer must demonstrate reasoning, comparison, or justification, not recall of definitions.

**S
1**

SCENARIO

You are a data analyst at a food delivery startup. The company has two years of order data sitting in spreadsheets but all decisions are still made by gut feeling. Leadership asks you to map out exactly what steps the team would follow to build a tool that predicts which restaurant zones will face peak demand between 7–9 PM — and they want to know where modern AI tools fit into that process.

Question: Walk through every stage of the Data Science Lifecycle as it applies to this problem. For each stage — Problem Definition, Data Collection, Data Preparation, Exploration, Modelling, and Evaluation — state what concrete deliverable that stage produces. Then identify two specific stages where Generative AI tools could meaningfully accelerate the work, explain the task they would assist with, and describe one risk of relying on AI output without human review at each of those stages.

SCENARIO
**S
2**

You are collaborating with two other developers on a food delivery project. One teammate is writing the data-cleaning script, another is building the reporting script, and you are building the order-tracking script. Every time any of you pushes code to the main branch, the others' work breaks. Your manager insists the main branch must always be in a working state.

Question: Explain how a Git branching strategy resolves this coordination problem without slowing the team down. Describe the exact sequence of Git commands each team member would use — from creating their own branch through to raising a Pull Request and completing a reviewed merge. Then describe one realistic merge conflict that could occur in this scenario and how you would resolve it.

SCENARIO
**S
3**

You are a data analyst automating delivery-partner commission payouts for a food delivery startup. The finance team wants the commission applied consistently across a growing list of delivery order records, and manually calculating it in a spreadsheet has started producing errors.

Question: Design a solution that uses a named function for the calculation logic and applies it across the full list using `map()` with a lambda. Explain why `map()` is more appropriate here than writing a second for loop. Then explain what `filter()` contributes if the finance team needs only the records above a given threshold for an audit, and how `functools.reduce()` could compute the overall total in a single expression.

SCENARIO
**S
4**

You are designing the account model for a food delivery platform with three types of users — Customer, DeliveryPartner, and Admin — each of which needs to show a different profile summary when `display_profile()` is called.

Question: Explain how a base class with inheritance and method overriding cleanly models this. What specific runtime problem occurs if someone writes a loop that iterates over a mixed list of all user objects and calls the `display` method on each one — when that method is only defined on some subclasses and not on the base class? Then explain how you would redesign the base class so the loop never crashes, regardless of which new subclass is added in future.

S
5

SCENARIO

You are integrating a third-party payment gateway API into the food delivery platform's Python backend. During testing, the API intermittently returns status codes 200, 401, 429, and 500 — and one teammate's script crashed and corrupted the output file when the API returned a 500 mid-write.

Question: Explain what each of these status codes means and what the script must do differently in response to each one. Then describe how a try-except-finally block around the file-write step would have prevented the corrupted output file — specifically what finally guarantees and why it runs even when an exception is raised.

 S
6

SCENARIO

You are cleaning a raw customer contact records file where phone numbers were entered inconsistently — some with extra spaces, some with typos, some in completely wrong formats — and the operations team needs a clean list of valid entries before it can be used.

Question: Explain how you would use Python's re module to validate and extract only the correctly formatted entries from this text. Describe the general structure of the regular expression pattern you'd use (in words, not just symbols) and explain why re.findall() is more appropriate here than re.search(). Then describe one real-world formatting edge case that a naive pattern would incorrectly reject or accept.

B
Section B — Practical Tasks

Complete all tasks. Task 1 uses Git (Bash / Terminal); Tasks 2–4 use Python 3. Tasks progress from foundational Python-for-Data-Science skills through to an integrative task that ties the Git workflow from Module 1 back into the Python work from Module 2. Submit a .txt command log with terminal screenshots for the Git task, and a .py file with output screenshot for each Python task.

Task 1: Food Delivery Project Repository Setup

Initialise a professional Git repository for a Python project with the correct folder structure, a configured .gitignore, and a portfolio-ready README.

- Create a directory called food-delivery-pipeline, initialise it as a Git repository, and create the following structure with placeholder .gitkeep files: src/, data/raw/, data/processed/, docs/, tests/ — then stage and commit everything with the message 'feat: initialise project structure'.
- Create a .gitignore that excludes: Python virtual environments (venv/, .env), compiled Python files (__pycache__/, *.pyc), log files (*.log), and local config/secrets files (config.ini, secrets.json).

- Write a README.md containing: project title 'Food Delivery Demand Analytics', a one-paragraph problem statement, a folder structure table, a setup instructions section, and a placeholder Usage section — commit with message 'docs: add README and .gitignore'.
- Create two test files — notes.log and data/raw/orders_sample.txt — then run git status and include a screenshot confirming notes.log does not appear as untracked while orders_sample.txt does, proving your .gitignore is working correctly.

Task 2: Food Delivery Record Cleaning Script

Write a Python script that cleans and validates a raw hardcoded list of restaurants using string methods, type casting, and conditional logic.

- Hardcode a list of at least 8 raw restaurants names with inconsistent casing, extra whitespace, and at least 2 invalid/empty entries.
- Use string methods (strip(), title() or lower(), replace()) to clean each valid entry, and skip/report any entry that is empty or purely numeric using a conditional check inside a loop.
- Hardcode an associated list of amount strings (e.g. 'Rs 150', '200.50') and convert them to floats, stripping the currency symbol, then print a running total using a for loop.
- Print a clean, formatted summary at the end: total valid records, total skipped, and the running total amount.

Task 3: Food Delivery Fee Calculator with Functional Tools

Build a fee calculator over a list of dictionary records using functions, lambda expressions, and map/filter/reduce.

- Create a list of at least 6 dictionaries, each representing a restaurant with at least 'id', 'name', and 'amount' keys (hardcoded data).
- Write a named function that applies a tiered commission based on 'amount' (define at least 3 tiers), then reuse it via map() with a lambda to compute a 'fee' for every record.
- Use filter() to isolate records where the computed fee exceeds a given threshold, and print them clearly labelled for audit purposes.
- Use functools.reduce() to calculate the total fee across all records in a single expression, and print it rounded to 2 decimal places.

Task 4: Food Delivery Record Manager with Persistence

Build a class-based record manager that saves and loads data to/from a JSON file, with proper error handling — and commit the result to your Task 1 repository.

- Define a Order class with attributes (order_id, restaurant_name, amount, delivery_time_minutes, is_delivered) and a to_dict() method for JSON serialisation, plus a from_dict() class method to rebuild an instance from a loaded dictionary.

- Implement `save_records(records, filepath)` that writes a list of these objects to `data/processed/records.json` using `json.dump()` inside a `with open()` block, and `load_records(filepath)` that reads them back and reconstructs the objects.
- Wrap the file operations in a `try-except-finally` block: catch `FileNotFoundError` specifically when loading (return an empty list instead of crashing), catch a generic `Exception` for anything else, and use `finally` to print an 'Operation complete' message regardless of outcome.
- Stage and commit your final script inside the Git repository from Task 1 using the message 'feat: add record manager with JSON persistence', and include a screenshot of git log showing at least 2 commits.

C

Section C — Mini Capstone Project

Mini Project: Food Delivery Live Data Pipeline — Console Tool

Objective:

Build a console-based Food Delivery Live Data Pipeline Tool that fetches mock records from a public API, processes and manages them using Python OOP and higher-order functions, persists results to disk using file handling, and is developed and submitted inside a properly structured Git repository — integrating the Git workflow from Module 1 with the Python skills from Module 2.

Your project must:

- The entire project must live inside a Git repository with the professional folder structure from Task 1 — the final submission must include at least 5 meaningful commits on the main branch with a feature branch merged in via a Pull Request, visible in the GitHub repository history.
- The program must be menu-driven with at least 4 options: (1) Fetch and display the top 10 mock restaurants from `https://jsonplaceholder.typicode.com/posts?_limit=10`, treating each record's id and title as the restaurant's id and name, (2) Add a new restaurant (stored as a class instance), (3) Calculate and display commission for all records using `map()` and `filter()`, (4) Save all record objects to `data/processed/records.json` and load them back — returning to the menu after each action.
- All data must be managed using a `Order` class with attributes appropriate to the domain (at minimum an id, name, amount, and status flag) and a `to_dict()` method for JSON serialisation — records must be stored as a list of class instances in memory for the session.
- The API fetch in option 1 must check the HTTP status code before calling `.json()` and handle both 200 (success) and any other code (print error, skip fetch gracefully) — the program must not crash on an API failure.
- Input validation must be applied for all numeric fields when adding a record — if the user enters a non-numeric value, the program must print a clear error message and re-prompt without crashing.

D

Section D — AI-Augmented Learning

STEP 1 • BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a Python program for a Food Delivery platform that:

- Fetches the first 5 records from https://jsonplaceholder.typicode.com/posts?_limit=5 (treating them as mock restaurants) and prints each record's id and title.
- Applies a tiered commission — 10% for orders up to Rs 200, 15% for Rs 201–Rs 500, and 20% above Rs 500 — to a hardcoded list of 6 order amounts using `map()` and a `lambda`, and prints each original value alongside its calculated result.
- Handles an API failure (any status code other than 200) by printing a descriptive error message and stopping execution — without raising an unhandled exception that crashes the script.
- Saves the 5 fetched records to a file called `data/processed/records.json` using `json.dump()` inside a `with open()` context manager, with an indent of 2 for readability.

STEP 2 • TEST & DEBUG (WITHOUT AI)

Then, working without AI, run the code and find at least one bug, limitation, or improvement in the AI's solution. Common issues to look for: missing `status_code` check before calling `.json()`, the tiered calculation using integer division (`//`) instead of float division (`/`), the output file path not matching the project folder structure, or `json.dump()` missing the `indent` parameter making the file unreadable. Fix the issue yourself.

SUBMIT

All 3 items required

- 1 The exact prompt(s) you gave the AI tool.
- 2 The AI's original code and your corrected version, clearly labelled.
- 3 A 3–4 line note explaining what was wrong in the AI's version, why it would fail or produce incorrect output, and what you changed to fix it.